Savitribai Phule Pune University
Centre For Distance and Online Education

SAVITRIBAI PHULE PUNE UNIVERSITY, PUNE

CENTRE FOR DISTANCE AND ONLINE EDUCATION

Program	**Master of Computer Application**	
Course	**MJ Programming from First Principles**	
Topic Name	**Standard data types: numbers, Boolean, character, tuple, list (practice session) Part 1**	
Topic Code	**-**	
Unit/Module No. & Name	**2**	**Standard Data Types**
Self-Learning Hours	**-**	

Topic Details

© SPPU | Original Content | All Rights Reserved Copying or Reposting Without Permission is Strictly Prohibited

OBJECTIVE

1. This paper explains five common data types used in beginner programming, which are numbers, Boolean values, characters or strings, tuples, and lists.

2. It uses simple English and an academic style so that undergraduate students can quickly and correctly review important ideas.

3. The main focus is on how each data type stores information, what operations are safe to perform on it, and how choosing the correct data type improves clarity, accuracy, and program design.

4. By the end of this reading, students should be able to identify all five data types, describe at least three properties of each type, and compare them using ideas like mutability, order, and grouping.

5. Students should also be able to explain why a programmer may choose a tuple instead of a list, or a Boolean instead of a number, when representing a real-world situation.

1.0 INTRODUCTION

1.1 Why Data Types Matter

In programming, every piece of information is stored as a value. A value can be a number, a true or false sign, a word, or a group of items. The computer needs rules to store that value in memory and to use it in operations. These rules are called data types. When a programmer chooses a data type, they also show what the value means. For example, using a Boolean for "paid" tells everyone the answer can only be true or false, not "maybe" or "partly."

1.2 Types as Academic Tools

In academic work, data types help students think clearly. They help students describe a problem, build a model, and test the model. A model is a simple version of a real system. For example, a student can model a store order using numbers for prices, Booleans for yes/no fields, strings for names, tuples for fixed records, and lists for collections that can change. When the type matches the meaning, the program is easier to understand and has fewer mistakes.

1.3 The Five Types in This Reading

This reading covers numbers, Boolean values, characters or strings, tuples, and lists. These types exist in many programming languages. The examples match Python style because Python is common in beginner courses, but the ideas work in other languages too. Numbers show quantities, Booleans show logical truth, strings show text, tuples store a fixed group of related values, and lists store an ordered collection that can grow, shrink, and change.

2.0 MEANING AND DEFINITION

2.1 Numbers

2.1.1 Meaning

Numbers represent quantity and measurement. They support arithmetic, comparisons, and many kinds of calculations. In most languages, there are different kinds of numbers. The most common are integers, which are whole numbers like 0, 7, and -12, and floating-point numbers, which can include a decimal point, like 3.14 or -0.5.

2.1.2 Definition

A number data type is a set of values that represent size or amount, along with operations such as addition, subtraction, multiplication, division, and comparison. In Python, int represents integers and float represents floating-point numbers. The exact range and precision depend on the language and system, especially for floats, because floats store values using a limited number of bits.

2.1.3 Key Properties

Numbers can be positive, negative, or zero. Integers are exact within their allowed range. Floating-point numbers are approximate and can have rounding errors. For example, adding 0.1 and 0.2 may not give exactly 0.3 because of how computers store decimal-like values in binary form. In academic work, it is important to remember that a computer's float may not behave like a perfect math value. Students should test important calculations and use a tolerance when comparing float results. A tolerance is a small allowed difference, such as 0.0001, used to decide whether two results are close enough.

2.2 Boolean Values

2.2.1 Meaning

Boolean values represent logical truth. They answer questions like "Is the door open?" or "Did the user log in?" In logic, the basic values are true and false. A Boolean supports decision making. If a condition is true, the program can do one action. If it is false, the program can do another action.

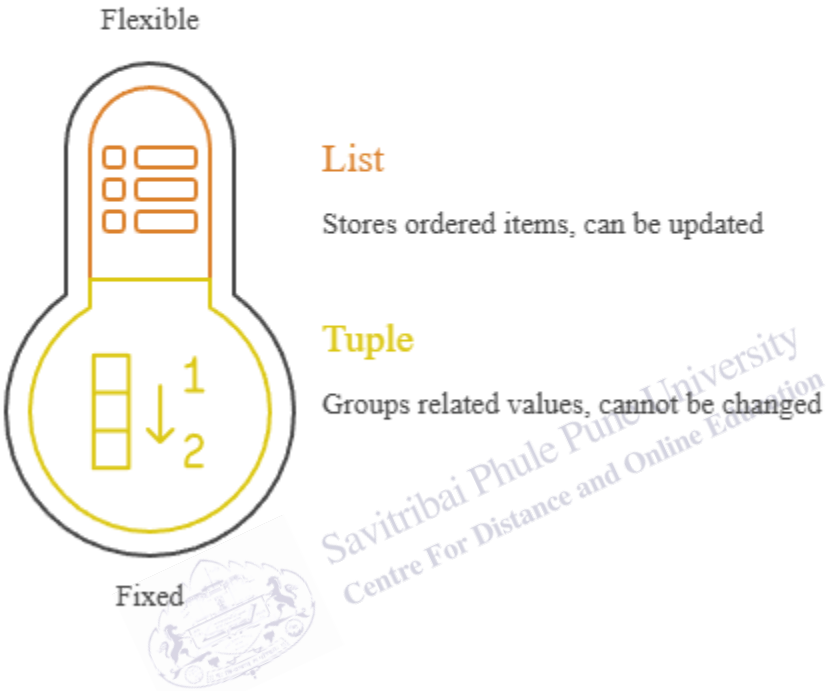
2.2.2 Definition

A Boolean data type has exactly two values, often written as True and False. Boolean operations include logical and, logical or, and logical not. These operations follow formal logic rules. For

example, True and False becomes False, while True or False becomes True. Comparison operations like “greater than” also produce Boolean results because they give yes/no answers.

Fig. 1

Data types range from fixed to flexible structures.



The image shows data types ranging from fixed to flexible structures. Lists are flexible, ordered, and updatable. Tuples are fixed, ordered groups of related values. A thermometer-style graphic visually compares mutability and structure in programming for beginner students learning concepts.

2.2.3 Key Properties

Booleans are often used in conditions, flags, and control flow. A flag is a variable that stores a state, such as logged_in or is_empty. Because a Boolean has only two values, it reduces confusion and makes testing easier. In many languages, Booleans are treated as different from numbers, even if conversion is possible. Keeping them separate helps code stay clear, especially when many people read and maintain the same program.

2.3 Characters and Strings

2.3.1 Meaning

Text is a common kind of data. A single letter such as A is called a character. A group of characters, such as “data,” is called a string. Some languages separate character and string types, while others use strings for both single and multiple characters. In Python, a string can store one character or many.

2.3.2 Definition

A character type represents one symbol from a character set such as ASCII or Unicode. A string type represents a sequence of characters. Common string operations include concatenation (joining strings), indexing (getting a character by position), slicing (getting a part of the string), and searching (finding a pattern).

2.3.3 Key Properties

Strings are ordered, so each character has a position. Many languages treat strings as immutable, meaning the string cannot be changed in place. Instead, a new string is created. This affects program design. For example, building a long string by joining many small parts one by one can be slow, so it may be better to collect parts in a list and join once at the end. Strings also need careful handling for case, spacing, and hidden characters like newline. Students should handle quotes, tabs, and escape characters carefully because these can change meaning and cause input problems.

2.4 Tuples

2.4.1 Meaning

A tuple groups a small set of related values into one unit. It is often used when the group has a fixed size and a clear order. For example, a point on a graph can be written as (x, y). A date can be written as (year, month, day).

2.4.2 Definition

A tuple is an ordered sequence of values that is usually immutable. In Python, a tuple is written with parentheses, such as (2, 3). A tuple can store values of different types, such as ("Riya", 19, True). Common operations include indexing, iteration, and unpacking. Unpacking means assigning each tuple value to a separate variable.

2.4.3 Key Properties

Tuples are ordered and can be nested inside other tuples or lists. Because tuples are often immutable, they can be used as keys in some data structures when their items are also immutable. Immutability also protects meaning. For example, if a tuple represents a GPS coordinate, changing it by mistake could cause serious errors. Tuples are also useful for returning multiple results from a function, such as (min_value, max_value) after scanning a list of numbers.

2.5 Lists

2.5.1 Meaning

A list stores an ordered collection of items that can change over time. Lists can grow, shrink, and be updated. This makes lists useful for data collected step by step, such as test scores, shopping cart items, or a queue of tasks.

2.5.2 Definition

A list is an ordered sequence of values that is usually mutable. In Python, a list is written with square brackets, such as [1, 2, 3]. Lists support indexing, slicing, adding items, removing items, and sorting. A list can hold different types, but in academic work it is usually better to keep list items similar so the program is easier to reason about and less likely to fail.

2.5.3 Key Properties

Lists are mutable, so an item can be changed in place. This makes updates efficient, but it also requires care. If two variables refer to the same list, a change through one variable changes what the other variable “sees.” This is called aliasing. To avoid confusion, students should learn when to make a copy of a list. Lists can also be nested to represent tables, grids, or tree-like data. For example, a gradebook can be a list of student records, where each record is a tuple like (name, scores).

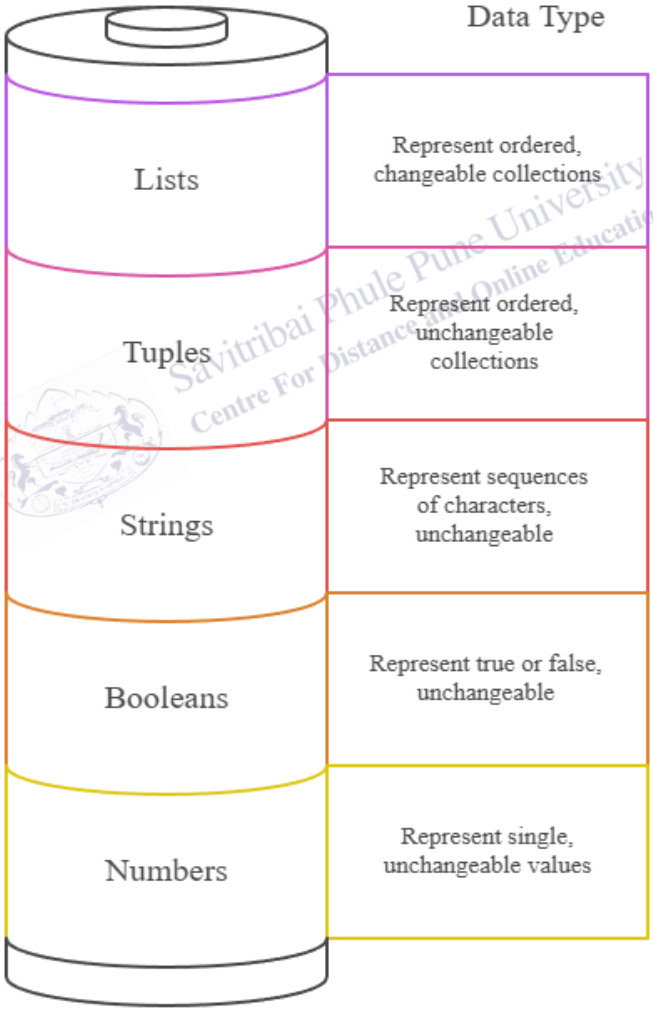
3.0 NATURE OR TYPE

3.1 Primitive and Composite Types

In many textbooks, numbers, Booleans, and characters are treated as primitive types, meaning they represent one basic value. Lists and tuples are composite types because they hold multiple values inside one structure. Composite types help model richer systems. For example, one number can store an age, but a tuple can store (name, age), and a list can store many such tuples.

Fig. 2

Understanding data types based on their mutability and structure.



The diagram explains data types by mutability and structure. Lists are ordered and changeable. Tuples are ordered but fixed. Strings store characters and cannot change. Booleans show true or false. Numbers hold single fixed values for beginner programming understanding students.

3.2 Ordered Types

All five types in this reading are ordered in useful ways, but "order" means different things in each case. For numbers, order means size, so 4 is less than 9. For strings, order means position in the text sequence. For tuples and lists, order means the first item is different from the second item. Order supports indexing, which lets a program select an item by position.

3.3 Mutable and Immutable Types

Mutability is an important idea in program design. A mutable object can change after it is created. An immutable object cannot change, so any "change" creates a new object. Lists are mutable. In many languages, tuples are immutable. Numbers and Booleans are also effectively immutable. Strings are usually immutable. This matters when values are shared. Sharing immutable values is safer because no one can change the value through another variable. Sharing mutable values can still be fine, but it needs careful planning and clear code.

3.4 Value Semantics and Reference Semantics

Some types behave like pure values. When you assign a number to another variable, you are effectively copying the value. With lists, assignment often copies a reference, not the full list. This means two variables can point to the same list object. Understanding this helps explain why a list can appear to "change" in two places at once. In lab work, students can test this by assigning one list to two names and then changing it through one name. This shows why copying methods and careful design are important.

3.5 Type Choice as Modeling Choice

In research and applied work, choosing a type shapes the model. A fixed record often fits a tuple. A changing collection often fits a list. A yes/no property fits a Boolean. A measured value fits a number. Labels and names fit strings. When types match meaning, programs are easier to test, read, and maintain. This also helps teamwork, because other students can quickly understand what each value is meant to represent.

4.0 EXAMPLES AND CASE STUDIES

4.1 Example: Tracking Library Books

Consider a small library system. Each book has a title, an author, a year, and a flag showing whether it is checked out. Title and author are strings. Year is a number. Checked out is a Boolean. One book record can be stored as a tuple: (title, author, year, checked_out). Because these four parts are fixed, a tuple works well. The library's full catalog can be a list of these tuples because the catalog can grow as new books are added. If the library later adds a field like genre, the tuple structure must be updated across the program.

4.2 Case Study: Student Score Analysis

Imagine a class where a teacher records quiz scores each week. Each score is a number. The teacher may add new scores, remove wrong entries, or update a score after a recheck, so a list is suitable. The teacher might store each student as a tuple (name, roll_number) because identity fields should stay stable. Then the teacher can store scores in a list such as [8, 9, 7, 10]. The average is found by adding scores and dividing by the count. The highest score is found by scanning the list and keeping the maximum value. The teacher can also store whether a student passed using a Boolean such as score >= 5. This case combines numbers for measurement, Booleans for decisions, tuples for identity, and lists for changing data.

4.3 Example: Password Rules

A website may require a password to be at least eight characters long and to contain a digit. The password is a string. The program checks the string length, which is a number. It also checks each character to see if it is a digit. Each check produces a Boolean. The final result can be stored in a Boolean called is_valid. This example shows how text and logic work together and how good naming supports clear meaning.

4.4 Case Study: Coordinates in a Map App

A map app uses latitude and longitude. Each is a number, often a float. A single location can be stored as a tuple (lat, lon) to keep the pair together and reduce accidental change. A route can be stored as a list of these tuples because a route is a sequence of locations. The list keeps order, so the first point is different from the last point. If the app stores a label for each stop, it can use a string and store (lat, lon, label).

4.5 Example: Simple Survey Data

Suppose a survey asks, "Do you like online classes?" The answer is yes or no, so a Boolean fits. Another question asks for age, so a number fits. Another asks for a favorite subject, so a string fits. One person's answers can be stored as a tuple (likes_online, age, favorite_subject). Many

responses can be stored as a list of these tuples. If researchers add a new question later, they must update the tuple structure and the code that reads it.

4.6 Mini Comparison: Tuple vs. List

Students often confuse tuples and lists because both store multiple items in order. The key difference is mutability. If the group should not change, a tuple communicates that rule. If the group should change, a list supports updates. Tuples often represent records where each position has a specific meaning, like (month, day, year). Lists often represent collections of similar items, like [quiz1, quiz2, quiz3]. Another important difference is sharing. Two names can point to the same list and changes will be shared. With tuples, sharing is safer because the values cannot be changed in place.

4.7 Case Study: Shopping Cart Modeling

A shopping cart shows type choice clearly. Each cart item has a product name, a unit price, a quantity, and a flag for whether the item is on sale. Name is a string. Price is a number, often a float. Quantity is an integer. On sale is a Boolean. One item can be stored as a tuple (name, price, quantity, on_sale) because the fields are fixed. The full cart can be a list of item tuples because items can be added or removed. The total cost is found by multiplying price by quantity for each item and adding the results. A receipt can be created by joining text parts into a final string.

4.8 Example: Sensor Readings Over Time

A temperature sensor takes a reading every minute. Each reading is a number, often a float. The sensor may also store a Boolean flag showing whether it is working. Each minute can be stored as a tuple (temp, ok). Over a day, the program stores many tuples in a list because the number of readings grows with time. The program can filter readings by keeping only those where ok is True. It can also compute an average while skipping readings where ok is False.

4.9 Case Study: Simple Game State

A quiz game uses all five types. The player’s name is a string. The score is a number. A Boolean called game_over can track whether the game has ended. Each question can be stored as a tuple (prompt, answer, points), where prompt and answer are strings and points is a number. The game holds many questions in a list so the set can be changed or reordered. When the player answers, the program compares the response string to the correct answer string, which produces a Boolean. If the answer is correct, the score increases. If there are no questions left, game_over becomes True.

4.10 Example: Attendance Tracking

In a classroom app, each day a student is present or absent, so a Boolean fits. The day can be stored as a string like “2026-01-31” or as a tuple (year, month, day). Weekly attendance can be stored as a list of Booleans such as [True, True, False, True, True]. The teacher can count absences by scanning the list and adding 1 each time the value is False.

1. Numbers represent quantity; integers are exact, but floats can have rounding errors.
2. Booleans store truth values and guide decisions using and, or, and not.
3. Strings store text as an ordered sequence of characters and are often immutable.
4. Tuples group a fixed set of related values and are commonly immutable.
5. Lists store an ordered collection that can grow, shrink, and be updated, so they are usually mutable.
6. Choosing the right type makes programs clearer, safer, and easier to test.

5.0 Keyword Touch Vocabulary Meaning

Keyword	Simple meaning	How it is used here
Data type	A kind of value	Tells what operations make sense
Integer	Whole number	Used for counts and exact values
Float	Decimal-like number	Used for measurements and approximations
Boolean	True or false value	Used for decisions and tests
String	Text value	Used for names, labels, and messages
Character	One text symbol	A single letter, digit, or sign
Tuple	Fixed ordered group	Used for records like (lat, lon)
List	Changeable ordered group	Used for collections like score sets
Mutable	Can change in place	Lists can be updated after creation
Immutable	Cannot change in place	Tuples and strings often behave this way
Tolerance	Small allowed difference	Helps compare floats safely
Aliasing	Two names, one list	Explains shared changes in lists